Topic 6 :Data Analysis and Visualisation

Learning Outcomes:

(a) Identify basic Pandas methods for data analysis (indexing, describe(), max(), min() and mean())

Topic 6 :Data Analysis and Visualisation

Table of Contents

- Information about dataset
- Explore Iris Dataset
- Indexing
- Indexing Operator
- Selecting column/columns with dot and bracket Operator

Datasource for Analysis

iris.csv

kaggle

Search

Create

Home

Competitions

Datasets

Models

Code

Discussions

Learn

More

Your Work

VIEWED

Indexing, Selecting ...

View Active Events

UCI MACHINE LEARNING AND 1 COLLABORATOR · UPDATED 8 YEARS AGO

3776 | New Notebook | Download (4 kB)

Iris Species

Classify iris plants into three species in this classic dataset

Data Card | Code (6893) | Discussion (31) | Suggestions (0)

About Dataset

The Iris dataset was used in R.A. Fisher's classic 1936 paper, The Use of Multiple Measurements in Taxonomic Problems, and can also be found on the UCI Machine Learning Repository.

It includes three iris species with 50 samples each as well as some properties about each flower. One flower species is linearly separable from the other two, but the other two are not linearly separable from each other.

The columns in this dataset are:

- Id

Usability
7.94

License
CC0: Public Domain

Expected update frequency
Not specified

Tags
Biology

link to site : https://www.kaggle.com/datasets/uciml/iris?select=Iris.csv

Information about datasets

Iris Species

3776 New Notebook

Data Card Code (6893) Discussion (31) Suggestions (0)

Iris.csv (5.11 kB)

Detail Compact Column 6 of 6 columns

About this file Add Suggestion

SQLite database containing the same data as Iris.csv

V(CM)	# SepalLengthCm Length of the sepal (in cm)	# SepalWidthCm Width of the sepal (in cm)	# PetalLengthCm Length of the petal (in cm)	# PetalWidthCm Width of the petal (in cm)	Species Species name
150	4.3 – 7.9	2 – 4.4	1 – 6.9	0.1 – 2.5	3 unique values
	5.1	3.5	1.4	0.2	Iris-setosa
	4.9	3.0	1.4	0.2	Iris-setosa
	4.7	3.2	1.3	0.2	Iris-setosa

- 5 columns
- 3 Species

1. Get to know the datasets using info() method

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

Use own file directory

```
df=pd.read_csv(file_path)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

Indicates whether there is an empty data or net

Datasets contain:

- 6 columns
- columns 0-4 are numerical columns.
- 150 rows

2. View the first 5 rows

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

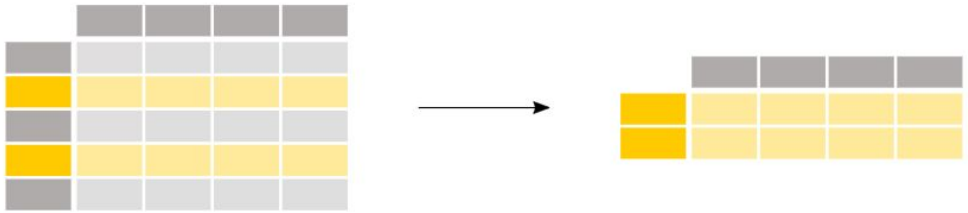
```
df=pd.read_csv(file_path)
```

```
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

3. Indexing from DataFrame

a. selecting specific columns



b. selecting specific rows

c. selecting specific rows and column

Indexing a pandas dataframe means **selecting** particular subsets of data (such as rows, columns, individual cells) from that dataframe

Indexing

Ways to Indexing

3a

Native Python Accessor
Column Selection Only

Attribute accessor
Single Column Only

- dot Operator

Indexing Operator
Single and Multiple Column

[ ] bracket Operator

3b & 3c

Pandas DataFrame Method
Column and Row Selection

Label based Indexing

loc[row_label,col_label**]**

Integer Based Indexing

iloc[row_indices,col_indices**]**

3a.1 dot operator

select single Column data based on header name

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df.Species
```

```
0         Iris-setosa
1         Iris-setosa
2         Iris-setosa
3         Iris-setosa
4         Iris-setosa
            ...
145    Iris-virginica
146    Iris-virginica
147    Iris-virginica
148    Iris-virginica
149    Iris-virginica
Name: Species, Length: 150, dtype: object
```

3a.2

[ ] operator

select Single Column data based on header name

```
[1] import pandas as pd
```

```
[3] # Store path to file in file_path variable
    file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
[4] df=pd.read_csv(file_path)
```

```
df['SepalLengthCm']
```

```
0      5.1
1      4.9
2      4.7
3      4.6
4      5.0
      ...
145    6.7
146    6.3
147    6.5
148    6.2
149    5.9
Name: SepalLengthCm, Length: 150, dtype: float64
```

3a.3

[ ] operator

Selecting Multiple Columns

To select multiple columns, the columns name is passed as a list.

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df[['SepalLengthCm','Species']]
```

	SepalLengthCm	Species
0	5.1	Iris-setosa
1	4.9	Iris-setosa
2	4.7	Iris-setosa
3	4.6	Iris-setosa
4	5.0	Iris-setosa
...	...	...
145	6.7	Iris-virginica
146	6.3	Iris-virginica
147	6.5	Iris-virginica
148	6.2	Iris-virginica
149	5.9	Iris-virginica

150 rows × 2 columns

3b. Using Pandas loc() method

Use to select **rows** and **columns** from a DataFrame using their labels.

How does `loc` work?

- For **rows**: You can specify the row or integer index label(s) you want to select.
- For **columns**: You specify the column label(s) you want to select

```
df.loc[row_labels,column_labels]
```

3b.1 Selecting one row

To select a single row, passed the row index value.

eg: row 100
It return single record with index of 100

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df.loc[100]
```

```
Id                            101
SepalLengthCm                 6.3
SepalWidthCm                  3.3
PetalLengthCm                 6.0
PetalWidthCm                  2.5
Species            Iris-virginica
Name: 100, dtype: object
```

3b.2 Selecting multiple rows

pass the row
start_index : end_index
to the loc operator

eg: loc[10:100]
It return the record with index of 10 until 100 **inclusive .**
[10:100] return 91 rows

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df.loc[10:100]
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
10	11	5.4	3.7	1.5	0.2	Iris-setosa
11	12	4.8	3.4	1.6	0.2	Iris-setosa
12	13	4.8	3.0	1.4	0.1	Iris-setosa
13	14	4.3	3.0	1.1	0.1	Iris-setosa
14	15	5.8	4.0	1.2	0.2	Iris-setosa
...	...	...	...	...	...	...
96	97	5.7	2.9	4.2	1.3	Iris-versicolor
97	98	6.2	2.9	4.3	1.3	Iris-versicolor
98	99	5.1	2.5	3.0	1.1	Iris-versicolor
99	100	5.7	2.8	4.1	1.3	Iris-versicolor
100	101	6.3	3.3	6.0	2.5	Iris-virginica

91 rows × 6 columns

3c.1 Selecting multiple rows of specific columns

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df.loc[10:100,['SepalLengthCm','SepalWidthCm']]
```

	SepalLengthCm	SepalWidthCm
10	5.4	3.7
11	4.8	3.4
12	4.8	3.0
13	4.3	3.0
14	5.8	4.0
...	...	...
96	5.7	2.9
97	6.2	2.9
98	5.1	2.5
99	5.7	2.8
100	6.3	3.3

91 rows × 2 columns

It return the record with index of
10 until 100 **inclusive**
and only columns

1. **SepalLengthCm**
2. **SepealWidthCm**

3c.2 Selecting specific rows and columns using relational/comparison operators

return

- SepalLengthCm
- SepalWidthCm
- Species

that has SepalLengthCm > 5.0

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df.loc[df.SepalLengthCm>5.0,['SepalLengthCm','SepalWidthCm','Speci
```

	SepalLengthCm	SepalWidthCm	Species
0	5.1	3.5	Iris-setosa
5	5.4	3.9	Iris-setosa
10	5.4	3.7	Iris-setosa
14	5.8	4.0	Iris-setosa
15	5.7	4.4	Iris-setosa
...	...	...	...
145	6.7	3.0	Iris-virginica
146	6.3	2.5	Iris-virginica
147	6.5	3.0	Iris-virginica
148	6.2	3.4	Iris-virginica
149	5.9	3.0	Iris-virginica

118 rows × 3 columns

Summary

- Indexing can create a subset of the dataframe
- Dot Operator could be used to select a column
- Bracket operator could be used select single and multiple columns.
- Using loc[ ] to select specific rows and columns using the index and columns label

Topic 6 :Data Analysis and Visualisation

Learning Outcomes:
(a) Identify basic Pandas methods for data analysis (indexing, describe(), max(), min() and mean())

Topic 6 :Data Analysis and Visualisation

Table of Contents

- row selection with iloc[ ] operator
- row and column selection with iloc[ ] operator
- Pandas method for basic statistical Summary
- Basic visualisation using pyplot module in Matplotlib

Recap : Indexing

Ways to Indexing

3a

Native Python Accessor
Column Selection Only

Attribute accessor
Single Column Only

- dot Operator

Indexing Operator
Single and Multiple Column

[ ] bracket Operator

3b & 3c

Pandas DataFrame Method
Column and Row Selection

Label based Indexing

loc[row_label,col_label**]**

Integer Based Indexing

iloc[row_indices,col_indices**]**

4. Using iloc()- Using Index instead of label/header name

It allows you to select specific rows and columns from a DataFrame using their **integer positions**. The i stand for **Integer** index.

How does `iloc` work?

- For rows: You specify the integer index position(s) you want to select.
- For columns: You specify the integer index position(s) you want to select.

```
df.iloc[row_index,column_index]
```

4b.1 Selecting one row

To select a single row, passed the row index value.

eg: row 100
It return single record with index of 100

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df.iloc[100]
```

```
Id                            101
SepalLengthCm                 6.3
SepalWidthCm                  3.3
PetalLengthCm                 6.0
PetalWidthCm                  2.5
Species             Iris-virginica
Name: 100, dtype: object
```

4b.2 Selecting multiple rows

pass the row **start_index: end_index** to the iloc operator

eg: loc[10:100]
It return the record start from index of 10 until 99.

index 100 is exclusive
[10:100] return 90 rows

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df.iloc[10:100]
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
10	11	5.4	3.7	1.5	0.2	Iris-setosa
11	12	4.8	3.4	1.6	0.2	Iris-setosa
12	13	4.8	3.0	1.4	0.1	Iris-setosa
13	14	4.3	3.0	1.1	0.1	Iris-setosa
14	15	5.8	4.0	1.2	0.2	Iris-setosa
...	...	...	...	...	...	...
95	96	5.7	3.0	4.2	1.2	Iris-versicolor
96	97	5.7	2.9	4.2	1.3	Iris-versicolor
97	98	6.2	2.9	4.3	1.3	Iris-versicolor
98	99	5.1	2.5	3.0	1.1	Iris-versicolor
99	100	5.7	2.8	4.1	1.3	Iris-versicolor

90 rows × 6 columns

4c.1 Selecting multiple rows and specific columns

It return rows of index 10 until 100 **exclusive** and only columns 1 until 3.

column index	Column
1	SepalLengthCm
2	SepalWidthCm
3	PetalLengthCm
4	**not included**

```python
import pandas as pd
```

```python
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```python
df=pd.read_csv(file_path)
```

```python
df.iloc[10:100,1:4]
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm
10	5.4	3.7	1.5
11	4.8	3.4	1.6
12	4.8	3.0	1.4
13	4.3	3.0	1.1
14	5.8	4.0	1.2
...	...	...	...
95	5.7	3.0	4.2
96	5.7	2.9	4.2
97	6.2	2.9	4.3
98	5.1	2.5	3.0
99	5.7	2.8	4.1

90 rows × 3 columns

Basic Statistical Analysis

1. describe() # return basic statistical summary
2. mean() # return average value of the selected columns
3. max() # return highest value in the selected columns
4. min() # return lowest value in the selected columns

Statistical analysis must only be applied to numerical columns

6. Describing Dataframe

`df.describe()`

Return statistical summary of all numerical columns

- **Count:** Number of non-null values in each column.
- **Mean:** Average value of each column.
- **Std:** Standard deviation of each column.
- **Min:** Minimum value in each column.
- **25%:** 25th percentile (first quartile) of each column.
- **50%:** 50th percentile (median) of each column.
- **75%:** 75th percentile (third quartile) of each column.
- **Max:** Maximum value in each column.

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df.describe()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

7a Max for all Species

```
df.max()
```

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df_max=df.loc[:,['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWidthCm']].max()
```

```
df_max
```

```
SepalLengthCm    7.9
SepalWidthCm     4.4
PetalLengthCm    6.9
PetalWidthCm     2.5
dtype: float64
```

1. : in the loc operator indicate we want to select all rows
2. 4 columns is selected
3. max() method return maximum value in the selected columns

7b Max for specific Species

```
df.max()
```

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df_max=df.loc[df.Species=='Iris-versicolor',['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWidthCm']].max()
```

```
df_max
```

```
SepalLengthCm    7.0
SepalWidthCm     3.4
PetalLengthCm    5.1
PetalWidthCm     1.8
dtype: float64
```

1. df.Species == ‘Iris-Versicolor’ is use to select only Iris-Versicolor rows.
2. 4 Columns is selected
3. max() method will return the maximum values for the selected columns

8a
Min for all Species

```
df.min()
```

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df_min=df.loc[:,['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWidthCm']].min()
```

+ Code + Text

```
df_min
```

```
SepalLengthCm    4.3
SepalWidthCm     2.0
PetalLengthCm    1.0
PetalWidthCm     0.1
dtype: float64
```

1. : in the loc operator indicate we want to select all rows
2. 4 columns is selected
3. min() method return min value in the selected columns

8b Min for specific Species

```
df.min()
```

```python
import pandas as pd
```

```python
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```python
df=pd.read_csv(file_path)
```

```python
df_min=df.loc[df.Species=='Iris-versicolor',['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWidthCm']].min()
```

```python
df_min
```

```
SepalLengthCm    4.9
SepalWidthCm     2.0
PetalLengthCm    3.0
PetalWidthCm     1.0
dtype: float64
```

1. df.Species == ‘Iris-Versicolor’ is use to select only Iris-Versicolor rows.
2. 4 Columns is selected
3. min() method will return the minimum values for the selected columns

9a Mean for all Species

```
df.mean()
```

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df_mean=df.loc[:,['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWidthCm']].mean()
```

```
df_mean
```

```
SepalLengthCm    5.843333
SepalWidthCm     3.054000
PetalLengthCm    3.758667
PetalWidthCm     1.198667
dtype: float64
```

1. : in the loc operator indicate we want to select all rows
2. 4 columns is selected
3. mean() method return mean value in the selected columns

9b Mean for a specific Species

```
df.mean()
```

```
import pandas as pd
```

```
# Store path to file in file_path variable
file_path='/content/drive/MyDrive/DataFrame/Iris.csv'
```

```
df=pd.read_csv(file_path)
```

```
df_mean=df.loc[df.Species=='Iris-versicolor',['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWidthCm']].mean()
```

```
df_mean
```

```
SepalLengthCm    5.936
SepalWidthCm     2.770
PetalLengthCm    4.260
PetalWidthCm     1.326
dtype: float64
```

1. df.Species == ‘Iris-Versicolor’ is use to select only Iris-Versicolor rows.
2. 4 Columns is selected
3. mean() method will return the mean values for the selected columns specific for Iris-Versicolor